

Genetic Algorithms

Introduction

- A genetic algorithm is a heuristic search and optimization technique inspired by the process of natural evolution.
- It's a type of evolutionary algorithm that uses mechanisms mimicking biological evolution such as selection, crossover, and mutation to solve complex optimization problems.
- The key idea behind GAs is to evolve a population of candidate solutions over time, selecting the fittest individuals and combining their attributes to produce new offspring that are better suited to the problem at hand.

Genetic Algorithm

1. Initialize population P with random individuals
2. Evaluate fitness of each individual in P
3. while (termination condition not met):
 4. Select parents from P based on fitness
 5. Apply crossover to selected parents to create offspring
 6. Apply mutation to offspring
 7. Evaluate fitness of offspring
 8. Replace least fit individuals in P with offspring
 9. Update P with new population
10. Return the best individual from P

Steps of a Genetic Algorithm

Initialization:

- Generate an initial population of individuals randomly.
- Each individual is represented by a chromosome (e.g., a string of bits or a sequence of numbers).

Evaluation:

- Calculate the fitness of each individual in the population using the fitness function.
- The fitness function measures how well the individual solves the problem.

Selection:

- Select individuals from the population to be parents based on their fitness. Higher fitness individuals have a higher chance of being selected.
- Common selection methods include roulette wheel selection, tournament selection, and rank-based selection.

Crossover (Recombination):

- Create offspring by combining the chromosomes of two parents.
- For example, with a single-point crossover, you might take the first half of one parent's chromosome and combine it with the second half of another parent's chromosome.

Mutation:

- Introduce random changes to individual chromosomes to maintain diversity in the population.
- For example, if chromosomes are binary strings, a mutation might flip a bit from 0 to 1 or vice versa.

Replacement:

- Form a new population by selecting some or all of the offspring and potentially some of the parents based on their fitness.
- This new population will then undergo the same process of selection, crossover, and mutation.

Termination:

- Repeat the evaluation, selection, crossover, and mutation steps until a termination condition is met.
- Termination conditions might include a maximum number of generations, a solution that meets a pre-defined fitness threshold, or convergence (i.e., when successive generations do not produce significantly better solutions).

Example: Solving the Traveling Salesman Problem (TSP)

- The Traveling Salesman Problem (TSP) is a classic optimization problem where the objective is to find the shortest possible route that visits a set of cities exactly once and returns to the starting city.

Step-by-Step Genetic Algorithm for TSP:

Step 1: Representation (Chromosome)

- **Chromosome:** Each chromosome represents a tour of cities. If there are n cities, the chromosome is an ordered list of n elements where each element corresponds to a city. The tour starts at a city, visits each city exactly once, and returns to the starting city.
- Example: For 5 cities (A, B, C, D, E), a chromosome might be represented as [A, C, B, E, D], and the tour would be [A, C, B, E, D, A].

Step 2: Initial Population

- **Population:** Generate an initial population of potential solutions (tours). These are typically random permutations of the cities.
- Example: With 5 cities, an initial population of 4 individuals might look like:
 - [A, B, C, D, E] [D, E, B, A, C] [C, A, D, E, B] [B, D, A, C, E]
- Each tour would be completed by returning to the starting city, so for [A, B, C, D, E], the full tour is [A, B, C, D, E, A].

Step 3: Fitness Function

- **Fitness Calculation:** The fitness of each chromosome is calculated based on the total distance of the tour. Shorter distances correspond to higher fitness.
- **Fitness Function:** $\text{Fitness} = 1 / \text{Total Distance}$
- Example: If the tour [A, B, C, D, E, A] has a total distance of 50 units, the fitness is $1/50=0.02$

- **Step 4: Selection Process:** Select pairs of chromosomes to be parents based on their fitness. Chromosomes with higher fitness have a higher chance of being selected.
- Common methods include:
 - **Roulette Wheel Selection:** Probability of selection is proportional to fitness.
 - **Tournament Selection:** Randomly select a subset of the population and choose the best individual from this subset.

- **Step 5: Crossover (Recombination)**
- Combine parts of two parent chromosomes to create offspring. Special crossover methods are used to ensure valid tours without repeated cities.
- Example: Suppose we have two parents:
 - Parent 1: [A, B, C, D, E]
 - Parent 2: [E, D, C, B, A]
- Using **Order Crossover (OX)**:
 - Randomly select a crossover segment from Parent 1: [B, C, D]
 - Copy this segment to the offspring.
 - Fill the remaining cities from Parent 2 in the order they appear, skipping cities already in the segment.
- Resulting offspring might look like:
 - Offspring: [E, B, C, D, A]
- The tour is completed by returning to the starting city: [E, B, C, D, A, E].

- **Step 6: Mutation**
- **Mutation Process:** Introduce random changes to individual chromosomes to maintain diversity in the population. This helps prevent premature convergence.
- Example: **Swap Mutation** might swap two cities in a tour:
 - Before Mutation: [A, B, C, D, E]
 - After Mutation: [A, D, C, B, E]
- The tour remains [A, D, C, B, E, A].

- **Step 7: Replacement**
- **Replacement Process:** Form a new population by selecting the best individuals from the current population and offspring. The new population is used for the next generation.
- This step ensures that better solutions are retained and used to produce even better offspring in subsequent generations.

- **Step 8: Termination**
- **Termination Condition:** The algorithm continues until a stopping condition is met, such as a maximum number of generations, a satisfactory fitness level, or convergence (no significant improvement over successive generations).
- Once the algorithm stops, the best chromosome in the population is considered the optimal or near-optimal solution.

Conclusion

- The genetic algorithm evolves a population of candidate tours over time, combining the fittest individuals to create new solutions.